



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Diseño e implementación de
una plataforma web para el
procesamiento de imágenes
mediante pipelines de
inteligencia artificial**



Presentado por Álvaro Pérez Mella
en Universidad de Burgos — 18 de mayo
de 2026

Tutores: Jose Luis Garrido Labrador y Jose
Miguel Ramirez Sanz



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. Jose Luis Garrido Labrador y D. Jose Miguel Ramirez Sanz, profesores del departamento de nombre departamento, área de nombre área.

Expone:

Que el alumno D. Álvaro Pérez Mella, con DNI 71956263G, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado “Diseño e implementación de una plataforma web para el procesamiento de imágenes mediante pipelines de inteligencia artificial”.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 18 de mayo de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. Jose Luis Garrido Labrador

D. Jose Miguel Ramirez Sanz

Resumen

Este Trabajo de Fin de Grado presenta el diseño e implementación de una aplicación web orientada al procesamiento de imágenes mediante modelos de inteligencia artificial. El sistema permite a los usuarios autenticados seleccionar flujos de análisis, subir imágenes, ejecutar modelos de visión por computador y obtener como resultado tanto una imagen procesada como información estructurada en formato JSON.

El proyecto parte de una primera fase experimental, centrada en validar la integración de modelos como YOLO y MediaPipe, y evoluciona hacia una plataforma más completa basada en Flask, SQLAlchemy y JWT. La aplicación incorpora una arquitectura modular organizada en controladores, servicios, modelos de datos y vistas web, lo que facilita la separación de responsabilidades y la ampliación futura del sistema.

Entre las funcionalidades implementadas destacan la gestión de usuarios y roles, el control de acceso mediante planes y licencias temporales de modelos de IA, la ejecución de pipelines formados por varias etapas de procesamiento, la generación de resultados temporales descargables mediante tokens y la existencia de una interfaz diferenciada para usuarios y administradores. Además, el proyecto incluye pruebas automatizadas orientadas a verificar el backend.

El resultado es una base funcional y extensible para una plataforma de análisis de imágenes mediante inteligencia artificial, preparada para ser ampliada con nuevos modelos, modos de ejecución y mejoras de despliegue.

Descriptores

Visión por computador, inteligencia artificial, procesamiento de imágenes, aplicación web, Flask, YOLO, MediaPipe, pipelines, SQLAlchemy, MariaDB, JWT, autenticación, pruebas automatizadas.

Abstract

This Final Degree Project presents the design and implementation of a web application for image processing using artificial intelligence models. The system allows authenticated users to select analysis workflows, upload images, execute computer vision models and obtain both a processed image and structured information in JSON format.

The project began with an experimental stage focused on validating the integration of models such as YOLO and MediaPipe, and later evolved into a more complete platform based on Flask, SQLAlchemy and JWT. The application follows a modular architecture organised into controllers, services, data models and web views, which improves separation of concerns and facilitates future extensions.

The implemented features include user and role management, access control through plans and temporary AI model licences, execution of multi-stage processing pipelines, generation of temporary downloadable results through tokens, and separate interfaces for regular users and administrators. The project also includes automated tests to verify the backend.

The result is a functional and extensible foundation for an artificial-intelligence-based image analysis platform, prepared to incorporate new models, execution modes and deployment improvements.

Keywords

Computer vision, artificial intelligence, image processing, web application, Flask, YOLO, MediaPipe, pipelines, SQLAlchemy, MariaDB, JWT, authentication, automated testing.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
1.1. Materiales adjuntos	2
2. Objetivos del proyecto	5
2.1. Objetivo general	5
2.2. Objetivos del software	6
2.3. Objetivos técnicos	7
2.4. Objetivos personales	8
3. Conceptos teóricos	11
3.1. Procesamiento de imágenes y visión por computador	11
3.2. Detección de objetos y estimación de puntos clave	12
3.3. Modelos utilizados: YOLO y MediaPipe	13
3.4. Pipelines y factoría de modelos	15
3.5. Integración en una aplicación web	16
3.6. Pruebas automatizadas en sistemas con modelos de IA	17
3.7. Síntesis	17
4. Técnicas y herramientas	19
4.1. Metodología y seguimiento del proyecto	19
4.2. Lenguaje y entorno de desarrollo	20

4.3. Visión por computador e inteligencia artificial	20
4.4. Desarrollo web y backend	21
4.5. Frontend e interfaz de usuario	22
4.6. Persistencia y configuración	22
4.7. Pruebas y validación	23
4.8. Diseño, documentación y herramientas auxiliares	23
4.9. Asistencia mediante IA generativa	24
4.10. Síntesis	24
5. Aspectos relevantes del desarrollo del proyecto	25
5.1. Inicio del proyecto y validación de la idea	25
5.2. Metodología de trabajo y evolución incremental	26
5.3. Demo inicial de procesamiento de imágenes	27
5.4. Arquitectura modular orientada a la escalabilidad	31
5.5. Modelo de datos como base de la extensibilidad	33
5.6. Pipelines de procesamiento y ejecución ramificada	34
5.7. Factoría de IA y launchers	36
5.8. Configuración por etapas y separación del código	37
5.9. Administración de pipelines sin modificar código	38
5.10. Usuarios, permisos y lógica de acceso	39
5.11. Gestión de resultados temporales y ciclo de vida de los datos	40
5.12. Validación de entradas, tratamiento de errores y robustez	41
5.13. Pruebas automatizadas y calidad del sistema	41
5.14. Datos iniciales y reproducibilidad de la demostración	43
5.15. Síntesis	43
6. Trabajos relacionados	45
6.1. Herramientas y plataformas relacionadas	45
6.2. Comparación con el proyecto desarrollado	46
6.3. Posicionamiento del TFG	47
7. Conclusiones y líneas de trabajo futuras	49
7.1. Conclusiones	49
7.2. Líneas de trabajo futuras	51
Bibliografía	53

Índice de figuras

5.1. Resultado de la ejecución de MediaPipe en modo hands , con representación de landmarks y conexiones sobre ambas manos detectadas.	28
5.2. Capturas asociadas a MediaPipe en modo pose . En (a) se muestra la interfaz de la demo con la imagen de entrada y los landmarks corporales detectados. En (b) se presenta la salida JSON generada, con información de landmarks, visibilidad y ruta de la imagen procesada.	29
5.3. Capturas asociadas al modelo YOLO. En (a) se observa la interfaz de la demo mostrando la imagen procesada con cajas delimitadoras sobre los objetos detectados. En (b) se presenta la salida JSON correspondiente, incluyendo detecciones, confianza, coordenadas y ruta de la imagen generada.	30
5.4. Organización arquitectónica inspirada en MVC con una capa adicional de servicios.	32
5.5. Flujo de ejecución de pipelines y resolución dinámica mediante la factoría de IA.	37

Índice de tablas

3.1. Comparación entre detección de objetos y estimación de puntos clave.	13
5.1. Decisiones orientadas a la escalabilidad funcional de la aplicación.	33
5.2. Resumen de bloques cubiertos mediante pruebas automatizadas.	42
6.1. Comparación entre trabajos relacionados y el proyecto desarrollado.	46

1. Introducción

En los últimos años, la inteligencia artificial aplicada a la visión por computador ha experimentado un importante crecimiento, especialmente en tareas como la detección de objetos, la estimación de pose, el reconocimiento de manos o el análisis automático de imágenes. Estas técnicas permiten extraer información útil a partir de una imagen y transformarla en resultados interpretables, tanto de forma visual como mediante datos estructurados.

Sin embargo, el uso de estos modelos no se limita únicamente a ejecutarlos de forma aislada desde un script. Para que puedan ser utilizados por usuarios no técnicos o integrados en otros sistemas, resulta necesario construir una aplicación que permita cargar imágenes, seleccionar el tipo de análisis deseado, ejecutar el procesamiento en el servidor y devolver los resultados de forma clara. Este planteamiento exige combinar conocimientos de desarrollo web, gestión de datos, autenticación, procesamiento de imágenes e integración de modelos de inteligencia artificial.

El presente Trabajo de Fin de Grado aborda el diseño e implementación de una plataforma web orientada al procesamiento de imágenes mediante modelos de inteligencia artificial. La aplicación permite a los usuarios autenticados seleccionar flujos de análisis, subir una imagen, ejecutar modelos como YOLO o MediaPipe y obtener como resultado una imagen procesada junto con información estructurada en formato JSON.

El proyecto ha evolucionado desde una primera demo técnica, cuyo objetivo era comprobar la viabilidad de ejecutar modelos de visión artificial desde un servidor web, hasta una aplicación más completa y modular. La versión actual incorpora una arquitectura basada en Flask, organizada en controladores, servicios, modelos de datos y vistas. Además, incluye autenticación mediante tokens, gestión de usuarios y roles, control de acceso

en función de planes o licencias, ejecución de pipelines de procesamiento, almacenamiento temporal de resultados y una interfaz diferenciada para usuarios y administradores.

Uno de los aspectos más relevantes del sistema es la utilización de pipelines de inteligencia artificial. En lugar de limitarse a ejecutar un único modelo sobre una imagen, la aplicación permite definir flujos formados por varias etapas ordenadas. Cada etapa se asocia a un modelo y a un modo de ejecución concretos, lo que permite combinar tareas como detección de objetos, recorte de personas o estimación de pose. Esta aproximación facilita la ampliación del sistema y permite construir análisis más complejos a partir de componentes reutilizables.

Desde el punto de vista funcional, el sistema cubre el flujo principal de uso: registro e inicio de sesión, acceso al panel de usuario, consulta de los pipelines disponibles, subida de una imagen, ejecución del análisis, visualización de resultados y descarga de los archivos generados mientras sigan disponibles. Junto a este flujo principal, se han incorporado funcionalidades complementarias como la gestión de planes, el alquiler temporal de modelos de inteligencia artificial, la consulta de compras, una guía de pipelines y un historial de ejecuciones con archivos temporales asociados.

Desde el punto de vista administrativo, la aplicación permite consultar usuarios, revisar ejecuciones, gestionar estados de cuenta y administrar pipelines. Estas funcionalidades no convierten el proyecto en una plataforma comercial completa, ya que no se integra una pasarela de pago real, pero sí permiten validar la lógica de acceso, contratación simulada y control de permisos necesaria para una evolución posterior.

1.1. Materiales adjuntos

Junto a esta memoria se entregan los siguientes materiales:

- Código fuente completo de la aplicación web.
- Anexos técnicos del proyecto.
- Ficheros de configuración utilizados por los modos de procesamiento.
- Script de inicialización de datos para preparar un entorno de demostración.

- Modelos locales necesarios para la ejecución de los modos de inteligencia artificial, cuando su tamaño y licencia permitan incluirlos en la entrega.
- Instrucciones de instalación y ejecución de la aplicación.
- Repositorio del proyecto.
- Vídeo o capturas de funcionamiento de la aplicación.

2. Objetivos del proyecto

El presente Trabajo de Fin de Grado tiene como finalidad desarrollar una aplicación web orientada al procesamiento de imágenes mediante modelos de inteligencia artificial. La idea principal es que un usuario pueda acceder a la plataforma, subir una imagen, seleccionar un flujo de análisis y obtener un resultado comprensible, tanto en forma de imagen procesada como mediante información estructurada.

Al inicio, el trabajo se planteó como una forma de comprobar si era viable integrar modelos de visión por computador dentro de un servidor web. A medida que el desarrollo avanzó, esa primera idea fue creciendo hasta convertirse en una aplicación más completa, con autenticación de usuarios, gestión de roles, control de acceso a modelos, persistencia de datos, ejecución de pipelines y una zona de administración.

Por este motivo, los objetivos del proyecto no se limitan únicamente a ejecutar un modelo de inteligencia artificial sobre una imagen. También se busca construir una base organizada y ampliable, que permita añadir nuevos modelos, definir distintos modos de ejecución y controlar qué usuarios pueden acceder a cada funcionalidad.

A partir de esta finalidad general, los objetivos se dividen en tres bloques: el objetivo general del proyecto, los objetivos funcionales del software y los objetivos técnicos necesarios para llevar a cabo su diseño e implementación.

2.1. Objetivo general

El objetivo general del proyecto es diseñar e implementar una aplicación web capaz de ejecutar flujos de procesamiento de imágenes mediante modelos

de inteligencia artificial, devolviendo al usuario una salida visual y una respuesta estructurada que permitan interpretar el resultado del análisis realizado.

Este objetivo incluye la integración de modelos como YOLO y MediaPipe, pero también la organización del sistema para que dichos modelos puedan utilizarse dentro de una aplicación web con usuarios, permisos, configuraciones y resultados temporales asociados.

2.2. Objetivos del software

Los principales objetivos funcionales del software son los siguientes:

- Permitir el registro, inicio de sesión y gestión básica de cuentas de usuario.
- Diferenciar entre usuarios normales y administradores mediante un sistema de roles.
- Ofrecer una interfaz web desde la que el usuario pueda acceder al panel principal de análisis.
- Permitir la carga de imágenes desde el navegador para su procesamiento en el servidor.
- Mostrar al usuario únicamente los pipelines y modelos a los que tenga acceso según su plan o sus licencias activas.
- Integrar distintos modelos de inteligencia artificial dentro de una misma aplicación, como YOLO y MediaPipe.
- Permitir la ejecución de pipelines formados por una o varias etapas de procesamiento.
- Devolver resultados comprensibles mediante una imagen procesada y datos estructurados en formato JSON.
- Gestionar los resultados generados de forma temporal, mediante enlaces de descarga asociados a tokens.
- Evitar la exposición directa de rutas internas del servidor al usuario final.

- Incorporar un historial de ejecuciones que permita consultar los análisis realizados mientras sus archivos asociados sigan disponibles.
- Incluir una zona de tienda o catálogo desde la que se puedan consultar modelos y planes disponibles.
- Permitir la simulación funcional de suscripciones y alquileres temporales de modelos de inteligencia artificial.
- Proporcionar una interfaz de administración para consultar usuarios, revisar ejecuciones y gestionar pipelines.
- Separar la lógica de interfaz web de los endpoints utilizados para la comunicación entre cliente y servidor, permitiendo ejecutar determinadas funcionalidades mediante peticiones HTTP.

2.3. Objetivos técnicos

Además de los objetivos funcionales, el proyecto plantea una serie de objetivos técnicos relacionados con la organización interna del sistema y con la forma en la que se integran sus distintos componentes:

- Diseñar una arquitectura modular que separe la lógica de presentación, los controladores, los servicios de procesamiento y el modelo de datos.
- Utilizar Flask como base para construir la aplicación y organizar sus rutas principales.
- Definir un modelo de datos capaz de representar usuarios, roles, planes, suscripciones, modelos de inteligencia artificial, modos de ejecución, pipelines, ejecuciones y archivos temporales.
- Aplicar autenticación mediante tokens JWT para proteger las rutas privadas de la aplicación.
- Incorporar una capa de persistencia mediante SQLAlchemy, facilitando la interacción entre la aplicación y la base de datos.
- Organizar la ejecución de modelos de inteligencia artificial de forma desacoplada, permitiendo incorporar nuevos modelos sin modificar la lógica general de procesamiento.
- Implementar un ejecutor de pipelines capaz de recorrer las etapas definidas y encadenar el resultado de una etapa con la siguiente.

- Gestionar configuraciones específicas para cada modo de ejecución mediante ficheros JSON.
- Controlar errores frecuentes, como imágenes inválidas, configuraciones incorrectas, falta de permisos o fallos durante el procesamiento.
- Mantener los archivos generados como recursos temporales, aplicando una política de expiración.
- Preparar datos iniciales de prueba que permitan demostrar el funcionamiento del sistema con usuarios, planes, modelos y pipelines ya configurados.
- Desarrollar pruebas automatizadas para comprobar el funcionamiento de la autenticación, la tienda, los modelos de datos, la factoría de inteligencia artificial, los launchers y la ejecución de pipelines.
- Mantener una estructura de proyecto clara, que facilite la comprensión del código y su posible despliegue posterior.

2.4. Objetivos personales

Además de los objetivos funcionales y técnicos, el proyecto ha perseguido varios objetivos personales relacionados con la formación adquirida durante el grado:

- Aplicar de forma integrada conocimientos de desarrollo web, bases de datos, ingeniería del software e inteligencia artificial.
- Profundizar en el desarrollo de aplicaciones backend con Python y Flask.
- Comprender cómo integrar modelos de visión por computador dentro de una aplicación web real.
- Diseñar una arquitectura modular que facilite la mantenibilidad y ampliación del sistema.
- Trabajar con una metodología incremental, documentando la evolución del proyecto mediante sprints.
- Reforzar el uso de pruebas automatizadas para validar funcionalidades críticas.

- Preparar una base técnica cercana a un entorno profesional, con separación de capas, persistencia, control de acceso y despliegue reproducible.

En conjunto, el proyecto busca construir una aplicación funcional que vaya más allá de una simple prueba aislada de modelos de inteligencia artificial. La solución desarrollada permite validar la integración de técnicas de visión por computador dentro de una aplicación web, incorporando además mecanismos de acceso, persistencia, administración, ejecución por etapas y pruebas automatizadas.

De esta forma, el resultado obtenido sirve como base para una plataforma ampliable de análisis de imágenes, en la que podrían incorporarse nuevos modelos, nuevos tipos de procesamiento y mejoras relacionadas con el despliegue o la explotación del sistema en un entorno más real.

3. Conceptos teóricos

Para comprender el desarrollo del proyecto es necesario introducir una serie de conceptos relacionados con la visión por computador, el procesamiento de imágenes y la integración de modelos de inteligencia artificial dentro de una aplicación web. El objetivo de este capítulo no es describir en profundidad todas las tecnologías utilizadas, sino presentar los fundamentos que permiten entender el problema abordado y las decisiones de diseño tomadas durante el desarrollo.

El sistema desarrollado combina dos ámbitos principales. Por una parte, utiliza modelos de visión por computador para extraer información de imágenes. Por otra, integra dichos modelos dentro de una plataforma web con usuarios, permisos, persistencia, pipelines y resultados temporales. Esta combinación hace necesario explicar tanto los conceptos asociados al análisis visual como los relacionados con la ejecución controlada de modelos dentro de una aplicación.

3.1. Procesamiento de imágenes y visión por computador

La visión por computador es una rama de la inteligencia artificial orientada a obtener información útil a partir de imágenes o secuencias de vídeo. Su finalidad no consiste únicamente en almacenar o mostrar imágenes, sino en interpretarlas computacionalmente para localizar objetos, reconocer patrones, identificar puntos relevantes o generar una descripción estructurada de una escena.

Desde el punto de vista interno, una imagen digital puede entenderse como una matriz de píxeles. Cada píxel contiene información numérica sobre intensidad o color y, en imágenes a color, suele representarse mediante varios canales. Esta representación matricial es importante porque los modelos de inteligencia artificial no procesan la imagen como la percibe una persona, sino como un conjunto de valores numéricos que deben adaptarse al formato esperado por cada modelo.

En el proyecto, el flujo básico de procesamiento comienza con la recepción de una imagen subida por el usuario. A continuación, el servidor valida el fichero, lo almacena temporalmente, lo transforma si es necesario y lo entrega al modelo correspondiente. Una vez finalizado el análisis, el resultado se convierte en una salida interpretable, normalmente formada por una imagen anotada y una respuesta estructurada en formato JSON.

Este proceso introduce una diferencia importante respecto a una prueba aislada ejecutada desde un script local. En una aplicación web no basta con llamar a un modelo sobre una imagen almacenada en el equipo del desarrollador. Es necesario controlar la subida del fichero, la ubicación de los archivos generados, la relación entre la ejecución y el usuario, la disponibilidad temporal de los resultados y la forma en la que estos se devuelven al navegador.

3.2. Detección de objetos y estimación de puntos clave

Dentro de la visión por computador existen diferentes formas de analizar una imagen. En este proyecto destacan dos enfoques: la detección de objetos y la estimación de puntos clave o *landmarks*.

La detección de objetos consiste en localizar elementos de interés dentro de una imagen y asignarles una clase. Su salida habitual incluye una caja delimitadora, una etiqueta y una puntuación de confianza. La caja indica la posición aproximada del objeto, la clase expresa qué tipo de elemento ha sido detectado y la confianza representa la seguridad asociada a la predicción.

La estimación de puntos clave aborda el problema desde otra perspectiva. En lugar de delimitar un objeto completo mediante una caja, localiza puntos significativos de una estructura. En el caso de una mano, estos puntos pueden corresponder a articulaciones y extremos de los dedos. En el caso de una pose corporal, pueden corresponder a hombros, codos, muñecas, rodillas

u otras partes del cuerpo. A partir de estos puntos es posible reconstruir visualmente una postura o una estructura anatómica.

Ambos enfoques son complementarios. La detección de objetos permite saber qué aparece en una imagen y dónde se encuentra, mientras que los *landmarks* permiten describir con más detalle la estructura interna de determinados elementos. Esta diferencia es relevante en el proyecto porque permite combinar modelos con salidas distintas dentro de una misma aplicación.

La Tabla 3.1 resume las diferencias principales entre ambos enfoques y muestra cómo se aplican dentro del proyecto.

Aspecto	Detección de objetos	Estimación de landmarks
Resultado principal	Caja delimitadora, clase y confianza.	Puntos clave y conexiones entre ellos.
Utilidad	Localizar objetos o personas en la imagen.	Describir la estructura de manos o cuerpo.
Representación visual	Rectángulos y etiquetas sobre la imagen.	Esqueleto o malla de puntos sobre la imagen.
Uso en el proyecto	Detección general y recorte de personas mediante YOLO.	Detección de manos y estimación de pose mediante MediaPipe.

Tabla 3.1: Comparación entre detección de objetos y estimación de puntos clave.

Como se observa en la Tabla 3.1, ambos enfoques producen salidas diferentes, pero compatibles. Esta compatibilidad es la que permite plantear pipelines en los que una primera etapa localiza regiones de interés y una etapa posterior realiza un análisis más detallado sobre ellas.

3.3. Modelos utilizados: YOLO y MediaPipe

El proyecto utiliza dos familias de modelos con objetivos diferentes: YOLO y MediaPipe. La elección de ambos responde a la intención de integrar técnicas complementarias dentro de una misma plataforma.

YOLO, acrónimo de *You Only Look Once*, es una familia de modelos de detección de objetos. Su principal característica es que realiza la localización

y clasificación de objetos en una única pasada sobre la imagen, lo que lo convierte en una opción adecuada cuando se buscan resultados visuales rápidos y fáciles de interpretar [33].

De forma general, un detector YOLO recibe una imagen, extrae características visuales mediante una red neuronal y predice posibles cajas delimitadoras junto con sus clases y niveles de confianza. Posteriormente se aplican filtros para eliminar detecciones poco fiables o redundantes. El resultado puede representarse de dos formas: gráficamente, mediante cajas dibujadas sobre la imagen, y estructuralmente, mediante datos como clase, confianza y coordenadas.

En este proyecto, YOLO se utiliza para la detección general de objetos y también para localizar personas dentro de una imagen. En este segundo caso, las cajas detectadas permiten generar recortes individuales que pueden ser utilizados por etapas posteriores del pipeline. Es importante señalar que este proceso no constituye una segmentación semántica mediante máscaras, sino un aislamiento basado en las coordenadas de las cajas delimitadoras.

MediaPipe, por su parte, es un framework de percepción desarrollado para construir soluciones de análisis multimodal, especialmente en tareas como manos, rostro o pose corporal [17, 5]. A diferencia de un detector de objetos convencional, MediaPipe resulta especialmente útil cuando interesa obtener una representación estructural mediante puntos clave.

En el proyecto se utiliza MediaPipe para los modos de manos y pose. En ambos casos se trabaja con modelos locales en formato `.task`, que permiten realizar la inferencia en el propio servidor. El resultado no se limita a indicar que existe una mano o una persona, sino que proporciona landmarks con coordenadas y, en algunos casos, información adicional como visibilidad o presencia.

La combinación de YOLO y MediaPipe permite que la aplicación no dependa de un único tipo de análisis. YOLO aporta capacidad de localización general mediante cajas, mientras que MediaPipe aporta análisis estructural mediante landmarks. Esta complementariedad es la base de algunos pipelines del proyecto, especialmente aquellos en los que primero se localizan personas y después se aplica un análisis más específico sobre la imagen o sobre los recortes generados.

3.4. Pipelines y factoría de modelos

Un pipeline puede entenderse como una secuencia ordenada de etapas que se ejecutan sobre una entrada. En el contexto de este proyecto, cada etapa representa una operación de procesamiento de imagen asociada a un modelo de inteligencia artificial y a un modo concreto de ejecución.

La utilización de pipelines permite superar la idea de ejecutar únicamente un modelo aislado. En lugar de limitarse a aplicar siempre una única inferencia sobre una imagen, el sistema puede definir flujos de análisis formados por varias etapas. Por ejemplo, una etapa puede detectar personas en una imagen, otra puede generar recortes a partir de esas detecciones y una etapa posterior puede aplicar estimación de pose sobre cada recorte.

Este planteamiento introduce una característica importante: el flujo no siempre es estrictamente lineal. Algunas etapas pueden generar una única salida, mientras que otras pueden producir varias. Si en una imagen aparecen varias personas, una etapa de recorte puede generar varios archivos, y cada uno de ellos puede ser procesado después de forma independiente. Por tanto, el sistema debe contemplar tanto ejecuciones secuenciales como ejecuciones ramificadas.

Para que este funcionamiento sea mantenible, la selección de modelos no debe quedar mezclada con la lógica principal de la aplicación. Si cada nuevo modelo obligara a modificar directamente el ejecutor de pipelines o los controladores web, el sistema sería difícil de ampliar. Por ello se utiliza un mecanismo de factoría.

Una factoría permite recibir un identificador de modelo y devolver el componente encargado de ejecutarlo. En el proyecto, esta idea se aplica mediante una factoría de inteligencia artificial y distintos *launchers*. Cada familia de modelos dispone de un componente especializado: YOLO tiene su propio lanzador para resolver sus modos de ejecución, y MediaPipe dispone de otro para sus modos de manos y pose.

Esta organización aporta desacoplamiento. El ejecutor de pipelines no necesita conocer los detalles internos de cada biblioteca ni llamar directamente a funciones concretas de YOLO o MediaPipe. Su responsabilidad es recorrer las etapas del pipeline, solicitar el motor adecuado y coordinar las entradas y salidas. De esta forma, el sistema queda preparado para incorporar nuevos modelos o modos sin rediseñar la arquitectura principal.

3.5. Integración en una aplicación web

La integración de modelos de inteligencia artificial dentro de una aplicación web requiere resolver aspectos que no aparecen cuando el modelo se ejecuta de forma aislada. Además del procesamiento visual, es necesario gestionar usuarios, permisos, datos persistentes, archivos generados y comunicación entre cliente y servidor.

En el proyecto, la aplicación web actúa como punto de entrada para el usuario. Desde el navegador se realizan acciones como registrarse, iniciar sesión, consultar pipelines disponibles, subir una imagen, ejecutar un análisis y visualizar los resultados. El backend recibe estas peticiones, valida los datos, comprueba permisos y coordina la ejecución del procesamiento.

La persistencia de datos resulta necesaria porque el sistema maneja información que debe conservarse más allá de una petición concreta. La base de datos representa usuarios, roles, planes, modelos de inteligencia artificial, modos de ejecución, pipelines, etapas, ejecuciones y archivos temporales. De esta forma, la aplicación no depende de configuraciones fijas en el código, sino de entidades relacionadas que reflejan la lógica del sistema.

El control de acceso es otro concepto clave. La autenticación permite identificar al usuario que realiza una petición, mientras que la autorización determina si puede ejecutar una acción concreta. En este proyecto, el acceso a determinados pipelines depende del rol del usuario, de su plan y de las licencias activas que tenga asociadas. Por tanto, la disponibilidad de un análisis no depende solo de que el modelo exista, sino también de que el usuario tenga permiso para utilizarlo.

También resulta importante la gestión de resultados temporales. El procesamiento de imágenes genera archivos que deben estar disponibles para el usuario durante un tiempo limitado, pero no necesariamente de forma indefinida. Para evitar exponer rutas internas del servidor, los archivos generados se asocian a tokens de descarga. Así, el usuario accede al resultado mediante un identificador temporal, mientras que el servidor mantiene el control sobre la ubicación real del archivo, su disponibilidad y su expiración.

Esta integración convierte el proyecto en algo más amplio que una demo de inteligencia artificial. El sistema no solo ejecuta modelos, sino que organiza su uso dentro de una aplicación con reglas de acceso, persistencia, trazabilidad y gestión temporal de recursos.

3.6. Pruebas automatizadas en sistemas con modelos de IA

Las pruebas automatizadas permiten comprobar que el sistema funciona de forma correcta y que los cambios introducidos durante el desarrollo no rompen funcionalidades existentes. En una aplicación web, estas pruebas pueden validar modelos de datos, controladores, servicios, autenticación, permisos y tratamiento de errores.

Cuando una aplicación integra modelos de inteligencia artificial aparece una dificultad adicional: ejecutar inferencias reales en cada prueba puede ser lento, costoso y dependiente de archivos externos. Además, algunos errores que interesa validar no tienen que ver con el modelo en sí, sino con la lógica de aplicación que lo rodea.

Por este motivo, en este tipo de proyectos resulta útil emplear técnicas de simulación o *mocking*. Mediante estas técnicas se sustituyen temporalmente determinadas dependencias por respuestas controladas. Así es posible comprobar que un controlador valida correctamente una petición, que una función responde ante una imagen inválida, que un fallo de escritura se propaga o que una etapa de pipeline se comporta de forma adecuada, sin depender siempre de una inferencia real.

En el proyecto, las pruebas automatizadas ayudan a validar el comportamiento de la autenticación, la tienda, los pipelines, la factoría de inteligencia artificial, los launchers, los modelos de datos y las funciones de procesamiento. Esto aporta confianza sobre el funcionamiento del sistema y permite tratar de forma más segura una aplicación que combina lógica web, persistencia, permisos y procesamiento de imágenes.

3.7. Síntesis

Los conceptos expuestos en este capítulo permiten entender la base teórica del proyecto sin entrar en una descripción exhaustiva de cada tecnología. La visión por computador proporciona el marco necesario para comprender cómo una imagen puede convertirse en una entrada procesable por modelos de inteligencia artificial. Dentro de ese ámbito, YOLO y MediaPipe representan dos enfoques complementarios: detección de objetos mediante cajas delimitadoras y análisis estructural mediante landmarks.

Sobre esa base se construye la idea de pipeline, que permite encadenar varias etapas de procesamiento y reutilizar la salida de una como entrada de

otra. La factoría de modelos y los launchers permiten ejecutar esos modelos de forma desacoplada, manteniendo la aplicación preparada para futuras ampliaciones.

Finalmente, la integración web añade conceptos propios de una aplicación real: autenticación, autorización, persistencia, control de acceso, gestión temporal de resultados y pruebas automatizadas. La combinación de todos estos elementos explica el alcance del TFG: no se trata únicamente de ejecutar modelos de inteligencia artificial, sino de construir una plataforma web modular capaz de organizarlos, controlarlos y ofrecer sus resultados de forma usable y trazable.

4. Técnicas y herramientas

En este capítulo se recogen las principales técnicas, tecnologías y herramientas utilizadas durante el desarrollo del proyecto. No se pretende realizar una descripción exhaustiva de cada una de ellas, sino justificar su papel dentro del sistema construido.

La selección de herramientas ha estado condicionada por el tipo de aplicación desarrollada: una plataforma web capaz de recibir imágenes, ejecutar modelos de visión por computador, gestionar usuarios y permisos, almacenar información persistente y validar su funcionamiento mediante pruebas automatizadas.

4.1. Metodología y seguimiento del proyecto

El desarrollo se ha organizado siguiendo un enfoque iterativo e incremental. Esta forma de trabajo ha permitido avanzar por fases, validar primero los elementos de mayor incertidumbre técnica y, posteriormente, incorporar nuevas funcionalidades sobre una base ya comprobada.

Para la planificación y seguimiento se ha utilizado Jira, organizando el trabajo en sprints y tareas. Esta herramienta ha permitido mantener una visión estructurada del avance del proyecto, registrar las actividades realizadas y relacionarlas con la planificación temporal del TFG [1].

La comunicación con los tutores se ha realizado principalmente mediante Microsoft Teams, utilizado para reuniones, seguimiento del estado del proyecto y revisión de decisiones relevantes durante el desarrollo [18].

El código fuente se ha gestionado mediante GitHub, lo que ha permitido mantener un histórico de cambios, respaldar el proyecto y facilitar el control de versiones durante su evolución [9].

4.2. Lenguaje y entorno de desarrollo

El lenguaje principal utilizado ha sido Python. Su elección resulta adecuada para este proyecto por la disponibilidad de bibliotecas orientadas a desarrollo web, inteligencia artificial, visión por computador, persistencia de datos y pruebas automatizadas [7].

El desarrollo se ha realizado principalmente sobre Linux Debian. Esta decisión ha permitido trabajar en un entorno cercano al que podría encontrarse en un servidor real, facilitando la gestión de dependencias, rutas, permisos y servicios asociados a la aplicación [24].

Como entorno de desarrollo se ha utilizado Visual Studio Code, apoyado por su terminal integrada y extensiones para Python. También se ha empleado DBeaver para inspeccionar visualmente la base de datos durante el diseño e implementación del modelo relacional [19, 3].

4.3. Visión por computador e inteligencia artificial

La parte de procesamiento visual constituye uno de los elementos centrales del proyecto. Para cubrir distintos tipos de análisis se han utilizado modelos y bibliotecas orientadas tanto a detección de objetos como a estimación de puntos clave.

- **YOLOv8 y Ultralytics:** se han utilizado para la detección de objetos sobre imágenes. En el proyecto se emplea una versión ligera del modelo, `yolov8n`, adecuada para obtener resultados visuales mediante cajas delimitadoras, clases detectadas y niveles de confianza [33].
- **MediaPipe:** Framework utilizado para el análisis mediante landmarks.
Justificación: MediaPipe se ha empleado para los modos de detección de manos y estimación de pose. A diferencia de YOLO, que trabaja principalmente con cajas delimitadoras, MediaPipe permite obtener puntos clave y conexiones anatómicas, proporcionando una representación más estructural del elemento analizado [17, 5].

En la implementación se utiliza la interfaz *Tasks API* de MediaPipe para cargar modelos locales en formato `.task`. Los modelos `hand_landmarker.task` y `pose_landmarker_full.task` se almacenan en la carpeta `models/mp`. Si no están disponibles, el sistema puede descargarlos la primera vez; una vez guardados, las ejecuciones posteriores se realizan de forma local, sin depender de una API remota para la inferencia.

- **OpenCV**: se ha utilizado como biblioteca de apoyo para leer imágenes desde disco, transformar espacios de color, redimensionar recortes, dibujar resultados y guardar imágenes procesadas [31].
- **NumPy**: se ha empleado como soporte para el tratamiento matricial de imágenes, especialmente en combinación con OpenCV y en pruebas automatizadas donde se generan imágenes simuladas [4].

Esta combinación de herramientas permite trabajar con dos tipos de resultados complementarios: detecciones mediante cajas delimitadoras y análisis estructural mediante landmarks. Gracias a ello, la aplicación puede ofrecer tanto salidas visuales sencillas de interpretar como información estructurada asociada al procesamiento.

4.4. Desarrollo web y backend

El backend de la aplicación se ha construido utilizando **Flask**. Se ha elegido este framework por su ligereza y flexibilidad, ya que permite comenzar con una demo sencilla y evolucionar hacia una aplicación modular organizada mediante controladores, servicios, modelos de datos y vistas [25].

La aplicación utiliza *Blueprints* de Flask para separar las rutas según su responsabilidad. De este modo, funcionalidades como autenticación, tienda, pipelines o administración se encuentran organizadas en módulos diferenciados, evitando concentrar toda la lógica en un único archivo.

Para la autenticación se ha empleado Flask-JWT-Extended. Esta biblioteca permite generar y validar tokens JWT, necesarios para proteger las rutas privadas y asociar cada petición a un usuario autenticado [32].

Werkzeug se ha utilizado en dos aspectos concretos: la generación y comprobación de hashes de contraseñas, y el tratamiento seguro de nombres de ficheros subidos por el usuario mediante `secure_filename` [26].

También se ha utilizado `python-dotenv` para cargar variables de entorno desde un fichero `.env`. Esto permite separar del código valores como claves secretas, configuración de JWT o cadena de conexión a la base de datos [16].

4.5. Frontend e interfaz de usuario

Aunque el núcleo técnico del proyecto se encuentra en el backend y en el procesamiento de imágenes, se ha desarrollado una interfaz web para facilitar el uso y la demostración del sistema.

La interfaz se ha construido con **YHTML**, **CSS** y **JavaScript**. HTML define la estructura de las vistas, CSS permite mantener una presentación visual uniforme y JavaScript se encarga de comunicar la interfaz con los endpoints del backend, gestionar respuestas y almacenar temporalmente el token de sesión en el navegador [21, 20, 22].

Las vistas se sirven mediante plantillas de Flask. En el proyecto se utilizan para separar páginas públicas, vistas de usuario autenticado y zona de administración [25].

4.6. Persistencia y configuración

La aplicación necesita almacenar información relacionada con usuarios, roles, planes, licencias, modelos disponibles, modos de ejecución, pipelines, ejecuciones y archivos temporales. Para ello se ha utilizado **YMariaDB** como sistema gestor de base de datos relacional [6].

El acceso a datos se ha implementado mediante **SQLAlchemy** y **Flask-SQLAlchemy**. Estas herramientas permiten representar las tablas como clases de Python y trabajar con las entidades del sistema de forma más integrada con el código de la aplicación [2].

En el entorno de pruebas se utiliza **SQLite** en memoria. Esta decisión permite ejecutar las pruebas de forma rápida y aislada, creando y destruyendo el esquema de datos sin afectar a la base de datos principal del proyecto [30].

La configuración de los modos de ejecución se ha separado parcialmente del código mediante ficheros **JSON**. Estos archivos recogen parámetros por defecto de modos como detección con YOLO, recorte de personas, detección de manos o estimación de pose, facilitando su modificación sin alterar directamente las funciones de procesamiento [13].

Además, el proyecto incluye un script de inicialización, `seed.py`, que permite preparar datos de demostración: usuarios, roles, planes, modelos de inteligencia artificial, modos y pipelines preconfigurados. Su finalidad es facilitar la puesta en marcha de un entorno de prueba o defensa.

4.7. Pruebas y validación

La validación del sistema se ha realizado mediante pruebas automatizadas con **Pytest**. Estas pruebas cubren distintas partes del proyecto, como autenticación, tienda, pipelines, modelos de datos, factoría de inteligencia artificial, launchers y funciones de procesamiento [27].

Para evitar que todas las pruebas dependan de modelos reales o de operaciones costosas sobre disco, se ha utilizado **unittest.mock**. Esta herramienta permite simular lecturas de imágenes, escrituras de archivos, respuestas de modelos y errores controlados, comprobando la lógica del sistema de forma más rápida y aislada [8].

También se ha empleado el cliente de pruebas de Flask, que permite simular peticiones HTTP directamente sobre la aplicación sin levantar un servidor real. Esto resulta útil para comprobar respuestas JSON, códigos de estado, validaciones y comportamiento de los controladores [25].

4.8. Diseño, documentación y herramientas auxiliares

Durante el diseño del sistema se han utilizado herramientas de apoyo para representar la estructura de datos y documentar la arquitectura del proyecto.

- **dbdiagram.io**: utilizado para trabajar de forma ágil sobre el modelo relacional antes de trasladarlo a SQLAlchemy y MariaDB [12].
- **Draw.io**: utilizado para elaborar diagramas de apoyo, especialmente representaciones visuales del modelo entidad-relación y otros esquemas del sistema [14].
- **LaTeX**: utilizado para redactar la memoria y los anexos del TFG, generar índices, gestionar referencias y mantener una presentación adecuada para la documentación académica [34].

4.9. Asistencia mediante IA generativa

Durante el desarrollo y la redacción se han utilizado herramientas de IA generativa, principalmente **ChatGPT y Gemini**, como apoyo para contrastar ideas, revisar explicaciones, plantear alternativas de diseño, depurar errores y generar borradores de texto [23, 10].

Su uso ha sido auxiliar. Las decisiones de análisis, implementación, validación y redacción final han sido revisadas y adaptadas dentro del propio proceso de desarrollo del TFG.

4.10. Síntesis

Las herramientas utilizadas cubren las distintas necesidades del proyecto. Python, Flask, SQLAlchemy y MariaDB forman la base de la aplicación web y su persistencia. YOLO, MediaPipe, OpenCV y NumPy permiten abordar el procesamiento de imágenes y la integración de modelos de visión por computador. Pytest, `unittest.mock` y el cliente de pruebas de Flask aportan soporte para la validación automatizada del sistema.

Junto a ellas, herramientas como Debian, Visual Studio Code, DBeaver, GitHub, Jira, Teams, dbdiagram.io, Draw.io y LaTeX han servido de apoyo para el desarrollo, seguimiento, diseño y documentación del proyecto. En conjunto, esta selección ha permitido construir una aplicación web modular, con usuarios, permisos, pipelines, resultados temporales y una base preparada para futuras ampliaciones.

5. Aspectos relevantes del desarrollo del proyecto

En este capítulo se recogen los aspectos más relevantes del desarrollo del proyecto. No se pretende realizar una descripción cronológica exhaustiva de todas las tareas realizadas, ya que esa información se detalla con mayor precisión en los anexos de planificación y seguimiento. El objetivo de este capítulo es explicar las decisiones técnicas que han condicionado el diseño de la aplicación y que aportan mayor valor al resultado final.

El aspecto más importante del proyecto no es únicamente la ejecución de modelos de inteligencia artificial sobre imágenes, sino la construcción de una aplicación web escalable desde el punto de vista funcional. Es decir, una aplicación preparada para añadir nuevos flujos de procesamiento, modificar pipelines existentes, incorporar nuevos modos de ejecución y gestionar el acceso de los usuarios sin tener que rehacer el núcleo del sistema.

Esta idea de escalabilidad ha guiado buena parte del desarrollo. La aplicación no se ha planteado como una demo cerrada con dos o tres botones fijos, sino como una plataforma en la que los modelos, los modos, los pipelines, los permisos y los resultados temporales quedan organizados mediante una arquitectura modular y una base de datos relacional.

5.1. Inicio del proyecto y validación de la idea

El punto de partida del TFG fue construir una aplicación web capaz de procesar imágenes mediante modelos de inteligencia artificial. Desde el

inicio se planteó que el sistema no debía limitarse a ejecutar un script local, sino que debía permitir a un usuario subir una imagen desde el navegador, seleccionar un análisis y obtener un resultado interpretable.

La primera decisión importante fue validar cuanto antes la parte de mayor incertidumbre técnica: la integración de modelos de visión por computador en un servidor web. Antes de desarrollar la lógica completa de usuarios, permisos, planes o administración, era necesario comprobar que el flujo básico funcionaba de extremo a extremo.

Ese flujo inicial estaba formado por los siguientes pasos:

1. Subida de una imagen desde el navegador.
2. Recepción y almacenamiento temporal en el servidor.
3. Selección del modelo y modo de procesamiento.
4. Ejecución del análisis mediante el motor correspondiente.
5. Generación de una imagen procesada.
6. Devolución de una respuesta visual y estructurada.

Esta primera validación permitió comprobar que el planteamiento era viable y, al mismo tiempo, identificar varios problemas que después influirían en la aplicación final: gestión de rutas, separación entre interfaz y backend, tratamiento de errores, generación de resultados y necesidad de desacoplar la selección del modelo de la lógica HTTP.

5.2. Metodología de trabajo y evolución incremental

El desarrollo se realizó siguiendo un enfoque iterativo e incremental, se trabajó mediante ciclos de avance, revisión y ampliación funcional. Esta forma de trabajo resultó adecuada porque el proyecto combinaba varias áreas diferentes: desarrollo web, visión por computador, bases de datos, autenticación, control de acceso, diseño de pipelines y pruebas automatizadas.

La evolución del proyecto puede resumirse en tres grandes fases:

- **Validación técnica inicial:** construcción de una demo para comprobar la ejecución de YOLO y MediaPipe desde una aplicación web.

- **Diseño de la aplicación escalable:** definición del modelo de datos, roles, planes, modelos de IA, modos, pipelines, ejecuciones y resultados temporales.
- **Consolidación funcional:** implementación de la aplicación completa, administración de pipelines, panel de usuario, control de acceso, pruebas automatizadas y datos iniciales de demostración.

Esta evolución fue importante porque evitó diseñar una plataforma completa sin haber validado antes la parte más crítica: la ejecución real de modelos sobre imágenes enviadas desde el navegador. Una vez comprobado ese flujo, el trabajo se centró en convertir la demo en una aplicación organizada, mantenible y ampliable.

5.3. Demo inicial de procesamiento de imágenes

La demo inicial tuvo como finalidad comprobar la viabilidad del procesamiento de imágenes desde un servidor web. Para ello se integraron dos familias de modelos: YOLO y MediaPipe. La elección de ambas permitió validar dos tipos de salida diferentes.

YOLO ofrecía detección de objetos mediante cajas delimitadoras, clases y niveles de confianza. Esta salida era adecuada para comprobar visualmente qué elementos reconocía el modelo y en qué posición de la imagen se encontraban.

MediaPipe, por su parte, permitía trabajar con landmarks en modos como manos y pose corporal. Esta salida era diferente a la de YOLO, ya que no se centraba en cajas, sino en puntos clave y conexiones anatómicas. Gracias a ello, la demo permitía probar no solo detección de objetos, sino también análisis estructural de una imagen.

La demo permitió validar tres decisiones que se mantuvieron después en la aplicación final:

- **Resultado visual como salida principal:** la imagen procesada permite comprobar rápidamente si el análisis ha funcionado.
- **JSON como salida complementaria:** los datos estructurados permiten conservar información técnica sobre detecciones, coordenadas, confianza o landmarks.

- **Separación entre interfaz y procesamiento:** la vista web no debía contener la lógica de inferencia, sino comunicarse con componentes especializados del backend.

Las Figuras 5.1, 5.2 y 5.3 muestran algunos resultados representativos de esta fase inicial.

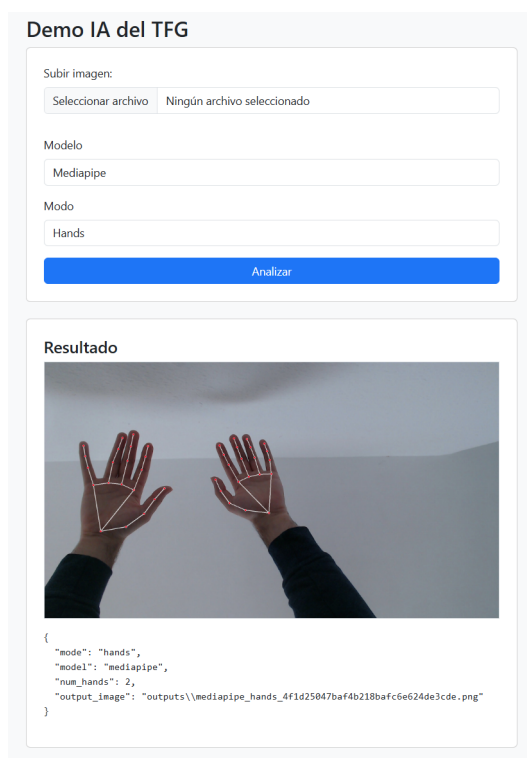
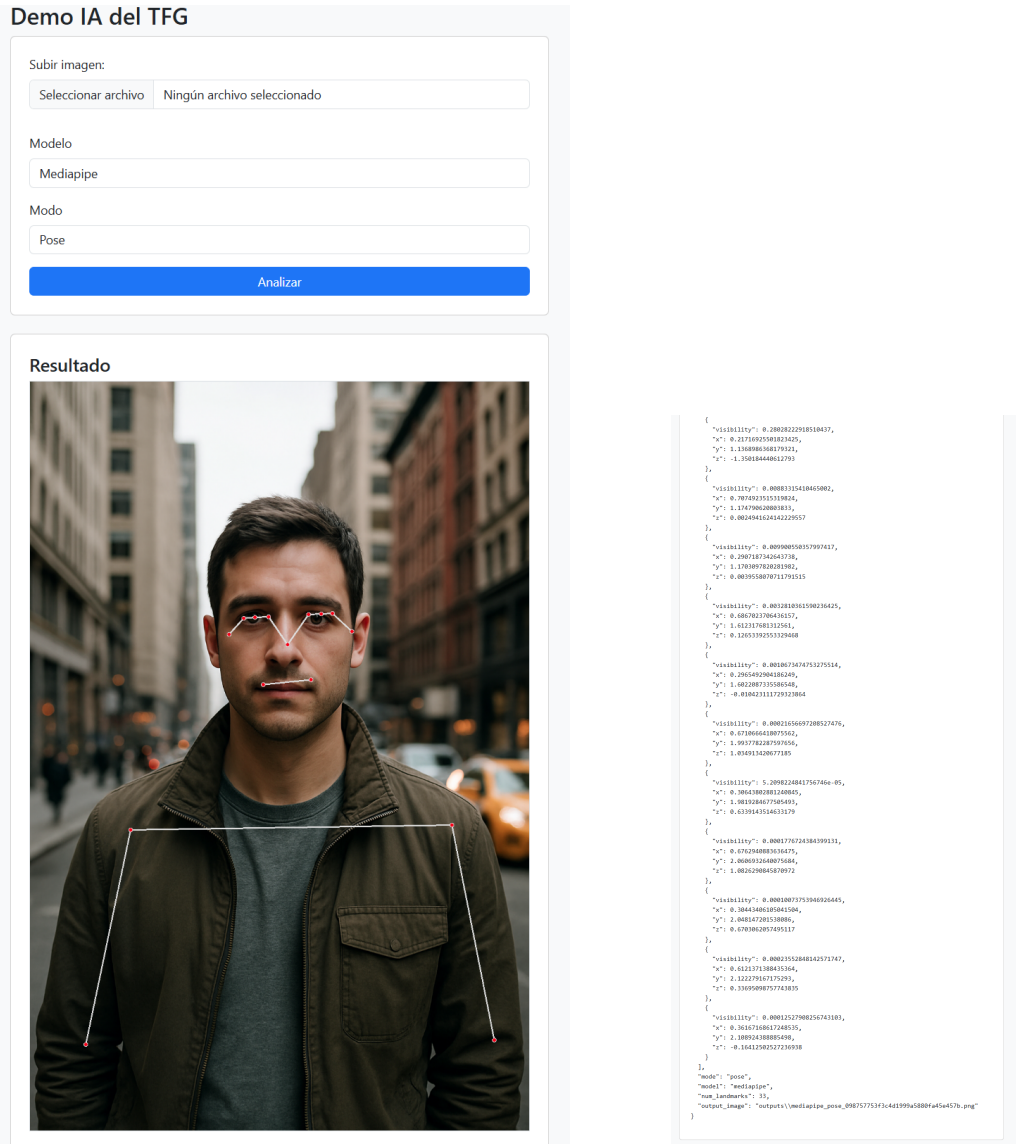


Figura 5.1: Resultado de la ejecución de MediaPipe en modo **hands**, con representación de landmarks y conexiones sobre ambas manos detectadas.

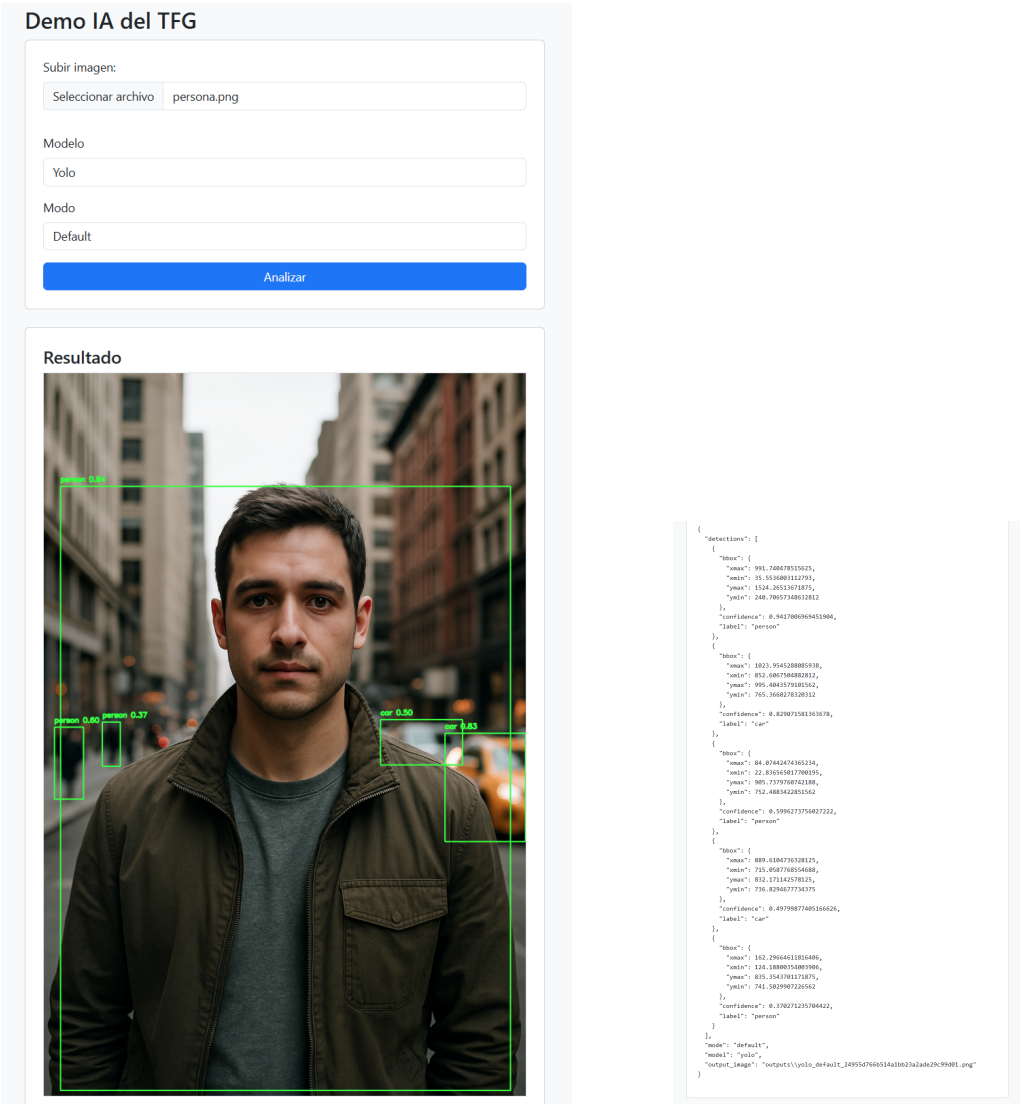
La demo no se consideró un producto final, sino una prueba técnica sobre la que construir la aplicación completa. Su valor estuvo en reducir incertidumbre y en mostrar qué partes del diseño debían formalizarse: modelo de datos, permisos, factoría de modelos, pipelines, almacenamiento temporal, trazabilidad y administración.



(a) Interfaz de la demo durante una ejecución de MediaPipe en modo pose.

(b) Salida JSON generada por MediaPipe en modo pose.

Figura 5.2: Capturas asociadas a MediaPipe en modo pose. En (a) se muestra la interfaz de la demo con la imagen de entrada y los landmarks corporales detectados. En (b) se presenta la salida JSON generada, con información de landmarks, visibilidad y ruta de la imagen procesada.



(a) Interfaz de la demo con ejecución de YOLO en modo **default**.

(b) Salida JSON asociada a la ejecución de YOLO.

Figura 5.3: Capturas asociadas al modelo YOLO. En (a) se observa la interfaz de la demo mostrando la imagen procesada con cajas delimitadoras sobre los objetos detectados. En (b) se presenta la salida JSON correspondiente, incluyendo detecciones, confianza, coordenadas y ruta de la imagen generada.

5.4. Arquitectura modular orientada a la escalabilidad

Uno de los aspectos más relevantes del desarrollo ha sido la consolidación de una arquitectura modular. Esta decisión se tomó para evitar que la lógica web, la lógica de negocio y la ejecución concreta de modelos de inteligencia artificial quedaran mezcladas.

La organización del proyecto se aproxima al patrón Modelo-Vista-Controlador, aunque no se trata de una aplicación MVC pura en sentido estricto. Se ha seguido una adaptación práctica de este patrón al ecosistema Flask, incorporando además una capa de servicios para separar la lógica de procesamiento de imágenes y de inteligencia artificial de las rutas HTTP.

En esta organización, los modelos se corresponden con las clases definidas mediante SQLAlchemy, responsables de representar entidades persistentes como usuarios, roles, planes, modelos de IA, modos, pipelines, ejecuciones y archivos temporales. Las vistas se materializan principalmente en las plantillas HTML de la aplicación y, en el caso de la API, en las respuestas JSON devueltas al cliente. Los controladores se organizan mediante *blueprints* de Flask, encargados de recibir las peticiones, validar entradas, comprobar permisos y coordinar la respuesta.

No obstante, la lógica de inferencia y orquestación de modelos no se ha ubicado directamente en los controladores. Para evitar controladores monolíticos, se ha incorporado una capa de servicios que contiene el ejecutor de pipelines, la factoría de inteligencia artificial y los *launchers* específicos de cada familia de modelos. Esta decisión permite que los controladores actúen como punto de entrada de la aplicación, mientras que los servicios concentran la lógica técnica de procesamiento.

Como se muestra en la Figura 5.4, la aplicación mantiene una organización inspirada en MVC, pero incorpora una capa de servicios para separar la lógica de procesamiento de imágenes y de inteligencia artificial de los controladores HTTP.

La arquitectura final se apoya en dos ideas principales. La primera es la factoría de aplicación de Flask, mediante la función `create_app`, encargada de inicializar la configuración, las extensiones y los módulos principales. La segunda es la factoría de inteligencia artificial, encargada de seleccionar el motor adecuado según el modelo y el modo de ejecución.

Esta separación permite distinguir responsabilidades:

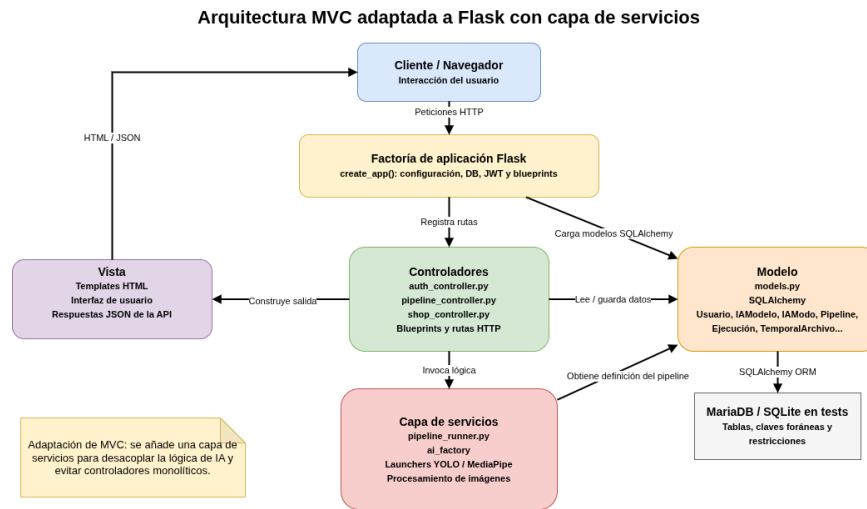


Figura 5.4: Organización arquitectónica inspirada en MVC con una capa adicional de servicios.

- **Modelos:** representan la información persistente mediante SQLAlchemy y definen la estructura de datos del sistema.
- **Vistas:** proporcionan la interfaz web mediante plantillas HTML y reciben también respuestas JSON desde la API.
- **Controladores:** reciben peticiones HTTP, validan entradas, comprueban permisos y coordinan la respuesta.
- **Servicios:** concentran la lógica de negocio, la orquestación de pipelines y el procesamiento de imágenes.
- **Factoría de IA:** selecciona el motor de procesamiento adecuado según el modelo solicitado.
- **Launchers:** encapsulan la ejecución concreta de cada familia de modelos.

Esta organización aporta escalabilidad funcional. La aplicación puede incorporar nuevos modelos, modos o pipelines sin modificar de forma significativa el flujo principal. Si se añade una nueva familia de modelos, el cambio se concentra en implementar su launcher, registrarlo en la factoría y definir sus modos. Si se desea crear un nuevo pipeline a partir de modelos y modos ya existentes, el administrador puede hacerlo desde la propia aplicación, sin modificar código fuente.

5.5. MODELO DE DATOS COMO BASE DE LA EXTENSIBILIDAD33

La Tabla 5.1 resume cómo se afrontan distintos tipos de ampliación dentro del sistema.

Necesidad de ampliación	Respuesta de la arquitectura
Crear un nuevo pipeline	Puede realizarse desde administración seleccionando etapas, modelos y modos disponibles.
Modificar un pipeline existente	El administrador puede editar sus etapas sin cambiar el código de ejecución.
Eliminar o deshabilitar un pipeline	Puede gestionarse desde la zona de administración, afectando a su disponibilidad para los usuarios.
Añadir un nuevo modo a un modelo existente	Requiere registrar el modo y asociarlo al launcher correspondiente, sin rediseñar el flujo principal.
Añadir una nueva familia de modelos	Requiere implementar un launcher específico y registrarlo en la factoría de IA.
Cambiar permisos de acceso	Se controla mediante roles, planes, licencias y estado de los pipelines/modelos.

Tabla 5.1: Decisiones orientadas a la escalabilidad funcional de la aplicación.

Esta arquitectura es uno de los puntos fuertes del proyecto, ya que demuestra que la aplicación no se limita a integrar librerías externas, sino que construye una capa propia para organizarlas, combinarlas y ejecutarlas de forma controlada.

5.5. Modelo de datos como base de la extensibilidad

El modelo de datos fue una de las partes más importantes del desarrollo. La aplicación no necesitaba guardar únicamente usuarios o credenciales, sino representar una lógica más amplia: roles, planes, suscripciones, alquileres de modelos, modelos de IA, modos de ejecución, pipelines, etapas, ejecuciones y archivos temporales.

La persistencia se diseñó con dos objetivos principales. Por un lado, permitir que la aplicación dejara de depender de datos fijos en el código.

Por otro, garantizar que las relaciones importantes quedaran controladas mediante restricciones de base de datos.

Una decisión especialmente relevante fue modelar por separado los modelos de inteligencia artificial y sus modos de ejecución. De esta forma, una IA puede tener varios modos asociados y cada etapa de un pipeline debe indicar tanto el modelo como el modo utilizado. Esta decisión evita representar el procesamiento mediante cadenas de texto libres y permite aplicar validaciones más estrictas.

La relación entre `IAModelo`, `IAModo` y `PipelineEtapa` es especialmente importante para la escalabilidad del sistema. La tabla de etapas utiliza una clave foránea compuesta sobre `id_modo` e `id_ia`, lo que impide guardar una etapa en la que el modo seleccionado no pertenezca al modelo indicado. Así, la coherencia del pipeline no depende únicamente de la interfaz o del controlador, sino que también queda reforzada en el modelo relacional.

Gracias a este diseño, los pipelines no se definen como código cerrado, sino como datos persistentes. Esta diferencia es fundamental: un pipeline puede crearse, editarse, deshabilitarse o eliminarse sin modificar las funciones de inferencia. El código mantiene la lógica general de ejecución, mientras que la base de datos almacena qué flujos existen y qué etapas los componen.

También se diferencié entre la ejecución lógica y los archivos físicos generados. La entidad `Ejecucion` permite conservar información histórica del procesamiento, como usuario, pipeline, estado, duración o errores. La entidad `TemporalArchivo`, en cambio, representa archivos concretos asociados a una ejecución, con ruta interna, token de descarga y fecha de expiración.

Esta separación permite conservar trazabilidad sin obligar a mantener indefinidamente todos los archivos generados. La ejecución puede seguir formando parte del historial aunque sus archivos temporales hayan expirado.

5.6. Pipelines de procesamiento y ejecución ramificada

La incorporación de pipelines es una de las aportaciones más importantes del proyecto. En la demo inicial, el usuario ejecutaba un modelo y un modo de forma aislada. En la aplicación final, el procesamiento se organiza mediante pipelines formados por etapas ordenadas.

Cada etapa indica qué modelo y qué modo deben ejecutarse. Esta estructura permite definir flujos simples, como una detección directa con YOLO o

5.6. PIPELINES DE PROCESAMIENTO Y EJECUCIÓN RAMIFICADA 35

una estimación de pose con MediaPipe, y también flujos más complejos en los que se combinan varias operaciones.

Un caso especialmente relevante es la ejecución ramificada. Algunas etapas pueden producir una única salida, pero otras pueden generar varias. Por ejemplo, el modo de recorte de personas puede detectar varios sujetos dentro de una imagen y generar un archivo independiente para cada uno. A continuación, una etapa posterior puede aplicar MediaPipe sobre cada recorte de forma individual.

Este comportamiento hace que el pipeline no sea siempre una secuencia estricta de una entrada y una salida. En determinados casos, una etapa produce varias ramas de procesamiento. Esta característica aporta valor al proyecto porque permite construir análisis más ricos que una inferencia aislada.

Desde el punto de vista de implementación, esta decisión obligó a tratar las rutas generadas como colecciones de archivos y no como un único resultado. También hizo necesario conservar información de cada etapa para que el usuario pudiera interpretar qué se había ejecutado y qué salidas se habían generado.

Algunos ejemplos de pipelines contemplados por el sistema son:

- Detección de objetos mediante YOLO.
- Recorte de personas mediante YOLO.
- Detección de manos mediante MediaPipe.
- Estimación directa de pose mediante MediaPipe.
- Detección de personas y posterior estimación de pose.
- Recorte de personas y análisis posterior de pose o manos sobre cada recorte.

Esta estructura muestra que el proyecto no consiste únicamente en llamar a modelos externos, sino en construir una capa propia de orquestación de análisis visual.

5.7. Factoría de IA y launchers

La factoría de inteligencia artificial es una pieza esencial para mantener la aplicación ampliable. Su objetivo es evitar que el ejecutor de pipelines o los controladores tengan dependencias directas con cada modelo concreto.

En lugar de escribir lógica específica para YOLO o MediaPipe en todos los puntos del sistema, la aplicación delega la selección del motor en una factoría. Esta factoría devuelve el launcher correspondiente, y cada launcher se encarga de resolver los modos propios de su familia de modelos.

La Figura 5.5 muestra el flujo interno seguido durante la ejecución de un pipeline. Cuando el usuario solicita un análisis, el controlador no ejecuta directamente ningún modelo de inteligencia artificial, sino que valida la petición, comprueba los permisos y delega la ejecución en `PipelineRunner`. Este componente consulta la definición del pipeline en la base de datos, obtiene sus etapas ordenadas y ejecuta cada una de ellas de forma secuencial.

En cada etapa, el sistema identifica el modelo de inteligencia artificial y el modo asociado. A partir de esa información, la factoría de IA selecciona el *launcher* correspondiente. De esta manera, el ejecutor de pipelines no necesita conocer los detalles internos de YOLO o MediaPipe, sino únicamente trabajar con una interfaz común de ejecución.

Esta organización resulta especialmente importante para la escalabilidad del proyecto. Si se añade una nueva familia de modelos, no es necesario modificar el controlador ni rehacer la lógica general del pipeline. El cambio se concentra en implementar un nuevo *launcher*, registrarlo en la factoría y asociar sus modos en la base de datos.

Además, el flujo contempla salidas ramificadas. Una etapa puede generar una única imagen procesada o varias imágenes intermedias, como ocurre en los recortes de personas. En ese caso, las rutas generadas pasan a ser la entrada de la siguiente etapa, permitiendo aplicar nuevos análisis sobre cada resultado intermedio.

El launcher de YOLO gestiona modos como detección o recorte de personas. El launcher de MediaPipe gestiona modos como manos o pose. Esta separación reduce el acoplamiento y permite que cada familia de modelos evolucione de forma independiente.

La existencia de una interfaz común para los launchers también aporta valor al diseño. Obliga a que las nuevas implementaciones respeten una forma común de ejecución. Gracias a ello, el sistema puede tratar distintos

5.8. CONFIGURACIÓN POR ETAPAS Y SEPARACIÓN DEL CÓDIGO

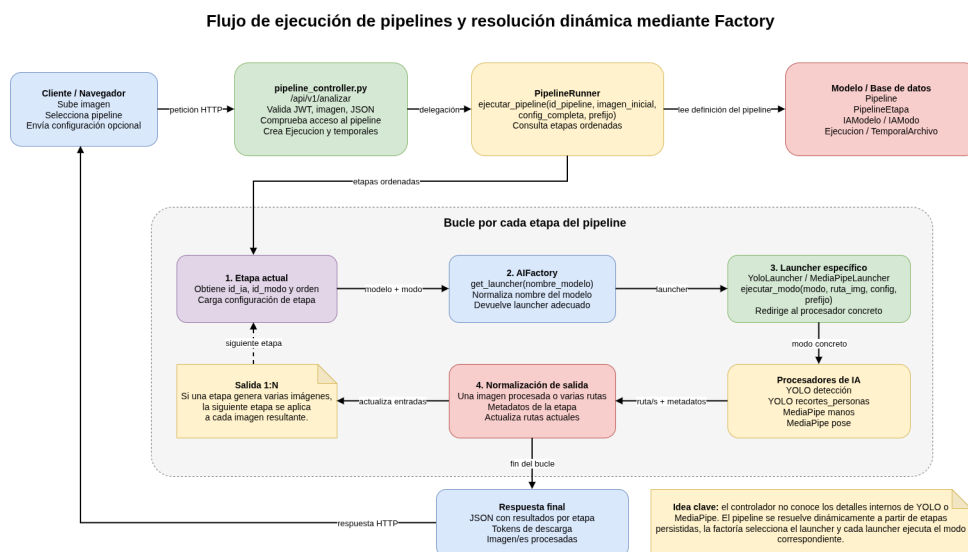


Figura 5.5: Flujo de ejecución de pipelines y resolución dinámica mediante la factoría de IA.

motores de IA de forma homogénea, aunque internamente cada uno utilice librerías, parámetros o estructuras de salida diferentes.

Esta decisión es clave para la escalabilidad del proyecto. Añadir una nueva familia de modelos no exige reescribir el ejecutor de pipelines ni los controladores principales. El cambio se concentra en un nuevo launcher y en su registro dentro de la factoría. Del mismo modo, añadir nuevos modos a una familia existente puede resolverse ampliando su launcher y registrando el modo correspondiente en la base de datos y en la configuración.

5.8. Configuración por etapas y separación del código

Otra decisión relevante fue separar parte de la configuración de los modos de ejecución mediante ficheros JSON. En lugar de fijar todos los parámetros directamente dentro del código, cada modo dispone de una configuración predeterminada que se carga automáticamente cuando el usuario selecciona un pipeline.

Esta configuración predeterminada permite que el sistema pueda utilizarse sin que el usuario tenga que conocer los parámetros internos de cada modelo. Es decir, la aplicación ofrece unos valores iniciales razonables para

cada modo de procesamiento, de forma que el análisis pueda ejecutarse directamente. No obstante, el usuario también puede modificar dicha configuración antes de lanzar la ejecución si desea alterar el comportamiento del modelo.

Esta posibilidad resulta especialmente útil porque algunos parámetros influyen directamente en el resultado obtenido. Por ejemplo, en un modelo de detección puede modificarse el umbral de confianza para controlar qué detecciones se aceptan como válidas. De forma similar, en modelos basados en landmarks pueden ajustarse valores relacionados con la confianza mínima de detección o seguimiento. Así, dos ejecuciones sobre una misma imagen y un mismo pipeline pueden producir resultados distintos si se modifican sus parámetros de configuración.

Desde el punto de vista de diseño, la aplicación distingue entre la configuración predeterminada asociada al modo y la configuración concreta utilizada en una ejecución. La primera actúa como plantilla inicial, mientras que la segunda representa los valores realmente aplicados cuando el usuario ejecuta el pipeline. Esta separación permite ofrecer una experiencia sencilla para usuarios que no quieran modificar parámetros y, al mismo tiempo, mantener flexibilidad para usuarios que deseen ajustar el análisis.

Esta decisión encaja con el objetivo de escalabilidad. Separar estos valores del código permite modificar o ampliar configuraciones sin alterar directamente las funciones de procesamiento. El código define cómo se ejecutan los modelos, la base de datos define qué pipelines existen y los ficheros JSON contienen valores configurables para los modos de procesamiento. De esta forma, el pipeline no solo es configurable en cuanto a sus etapas, sino también en cuanto al comportamiento concreto de cada etapa durante la inferencia.

5.9. Administración de pipelines sin modificar código

Una de las funcionalidades más relevantes desde el punto de vista de escalabilidad, fue la gestión de pipelines desde la zona de administración. Esta funcionalidad permite que un administrador pueda crear, editar, deshabilitar o eliminar pipelines desde la propia interfaz de la aplicación.

Este aspecto es importante porque evita que cada nuevo flujo de análisis tenga que programarse manualmente. En una versión más rígida del sistema, añadir un pipeline habría supuesto modificar código, crear una nueva ruta

o escribir una función específica. En la versión actual, los pipelines se construyen a partir de modelos y modos existentes registrados en la base de datos.

La interfaz de administración obtiene el catálogo de modelos y modos disponibles. A partir de esa información, el administrador puede definir las etapas del pipeline, su orden y su descripción. Antes de guardar el pipeline, el backend valida que las etapas sean correctas, que los modelos y modos existan, que estén habilitados y que cada modo pertenezca realmente al modelo seleccionado.

Esta validación es doble. Por un lado, se realiza desde la lógica de aplicación, devolviendo errores claros si una etapa no es válida. Por otro, se refuerza desde la base de datos mediante la clave foránea compuesta entre modo e IA. De esta manera, incluso aunque llegara una petición incorrecta, el modelo relacional impediría persistir combinaciones incoherentes.

Esta funcionalidad convierte la aplicación en una plataforma más flexible. Los pipelines dejan de ser una lista fija escrita en el código y pasan a ser configuraciones gestionables desde administración. Esto facilita la evolución del sistema y permite adaptar los flujos disponibles sin modificar la lógica principal de procesamiento.

5.10. Usuarios, permisos y lógica de acceso

Otro aspecto relevante del proyecto es la incorporación de una lógica de acceso a modelos y pipelines. El sistema distingue entre usuarios normales y administradores, y permite controlar qué funcionalidades puede utilizar cada tipo de usuario.

La aplicación incorpora autenticación mediante tokens JWT. Esto permite proteger rutas privadas y asociar cada petición a un usuario autenticado. A partir de esa identidad, el sistema puede comprobar roles, plan activo y licencias temporales de modelos.

La lógica de planes y alquileres añade una capa de complejidad sobre una demo convencional de inteligencia artificial. Un usuario con plan Pro puede acceder a todos los pipelines públicos mientras su suscripción esté activa además de poder mantener los archivos de sus ejecuciones durante una semana. En cambio, un usuario básico accede a los pipelines en función de las IAs que tenga alquileres y vigentes en ese momento.

Esta decisión permite simular un escenario cercano a una plataforma real, aunque sin integrar una pasarela de pago. La tienda y la guía de pipelines

no se limitan a mostrar modelos disponibles, sino que ayudan al usuario a entender qué modelos necesita para ejecutar cada flujo y qué acceso tiene en ese momento.

El control de acceso también está relacionado con la escalabilidad. Si se añaden nuevos pipelines o modelos, estos pueden integrarse dentro de la lógica existente de planes, licencias y roles. Por tanto, el sistema no solo permite ampliar los flujos de análisis, sino también controlar quién puede utilizarlos.

5.11. Gestión de resultados temporales y ciclo de vida de los datos

La gestión de archivos generados fue uno de los puntos que más evolucionó respecto a la demo inicial. En una primera prueba habría sido suficiente con guardar una imagen procesada y devolver su ruta. Sin embargo, en una aplicación con usuarios, historial y permisos, exponer rutas internas del servidor no resulta adecuado.

Por este motivo se incorporó una gestión de resultados temporales mediante tokens de descarga. Cada archivo generado puede quedar asociado a una ejecución y disponer de un token que permite acceder a él sin revelar su ubicación real en el servidor.

Esta decisión aporta varias ventajas:

- Evita exponer rutas internas de la aplicación.
- Permite asociar cada archivo a una ejecución concreta.
- Facilita controlar la expiración de resultados.
- Permite mostrar al usuario si un archivo sigue disponible.
- Separa el historial lógico de la conservación física de archivos.

Además, el sistema contempla tareas de mantenimiento relacionadas con el ciclo de vida de los datos. Estas tareas permiten eliminar archivos expirados, renovar suscripciones o alquileres cuando corresponde y anonimizar usuarios dados de baja tras un periodo determinado.

Este punto resulta relevante porque demuestra que el proyecto no solo contempla el caso ideal de uso, sino también qué ocurre después de la

ejecución: cuánto tiempo se conservan los resultados, cómo se accede a ellos, cuándo dejan de estar disponibles y cómo se mantiene la coherencia de los datos.

5.12. Validación de entradas, tratamiento de errores y robustez

Durante el desarrollo se prestó atención al tratamiento de errores y a la validación de entradas. En una aplicación que procesa archivos enviados por usuarios, este aspecto es importante porque pueden producirse situaciones no válidas: imágenes corruptas, extensiones no permitidas, configuraciones incorrectas, ausencia de permisos, modelos no disponibles o fallos al escribir resultados en disco.

Una decisión concreta fue comprobar explícitamente el resultado de operaciones de escritura con OpenCV. Algunas funciones, como `cv2.imwrite`, pueden devolver `False` si no consiguen guardar el archivo, sin lanzar necesariamente una excepción. Por ello, el código debe comprobar ese valor y transformar el fallo en un error controlado.

También se validan parámetros de configuración antes de ejecutar los modelos. Esto evita que una configuración mal formada llegue directamente al motor de inferencia y produzca un fallo más difícil de interpretar. En el caso de pipelines, se valida que las etapas sean coherentes, que los modelos y modos existan y que estén habilitados.

Estas validaciones no siempre son visibles desde la interfaz, pero mejoran la robustez de la aplicación y facilitan la depuración. Además, en estas situaciones se han incorporado a las pruebas automatizadas para comprobar que el sistema responde correctamente ante errores.

5.13. Pruebas automatizadas y calidad del sistema

La incorporación de pruebas automatizadas ha sido una parte importante del desarrollo. En un proyecto que combina aplicación web, base de datos, permisos, procesamiento de imágenes y modelos de inteligencia artificial, las pruebas permiten validar el comportamiento del sistema sin depender únicamente de comprobaciones manuales.

Las pruebas se han realizado con Pytest y utilizan una base de datos SQLite en memoria para aislar el entorno de pruebas de la base de datos principal. Además, se emplean técnicas de *mocking* para simular dependencias pesadas o externas, como modelos de inteligencia artificial, lectura y escritura de imágenes o errores controlados.

Esta estrategia permite comprobar la lógica del sistema sin ejecutar inferencias reales en cada prueba. Esto es importante porque los modelos pueden ser pesados, depender de archivos externos o hacer que las pruebas sean demasiado lentas.

La Tabla 5.2 resume los principales bloques de pruebas realizados.

Bloque probado	Aspectos validados
Autenticación y usuarios	Registro, inicio de sesión, perfil, cambio de contraseña, roles y baja lógica.
Tienda y acceso	Planes, alquileres, renovaciones, compras programadas y disponibilidad de IAs.
Pipelines	Listado, configuración, ejecución, historial, resultados y archivos temporales.
Administración de pipelines	Creación, edición, eliminación y validación de etapas, modelos y modos.
Factoría de IA y launchers	Selección del motor adecuado y ejecución de modos de YOLO y MediaPipe.
Procesamiento de imágenes	Validación de imágenes, configuraciones, recortes, manos, pose y fallos de escritura.
Mantenimiento	Expiración de temporales, renovación de servicios y anonimización de cuentas.

Tabla 5.2: Resumen de bloques cubiertos mediante pruebas automatizadas.

El valor de estas pruebas no está únicamente en cubrir casos correctos. También se comprueban situaciones de error, como imágenes inexistentes, configuraciones corruptas, usuarios sin permisos, archivos expirados, fallos de escritura o excepciones en operaciones de base de datos. Esto ayuda a construir una aplicación más fiable y preparada para comportamientos reales.

5.14. Datos iniciales y reproducibilidad de la demostración

Para facilitar la puesta en marcha del sistema se incorporó un script de inicialización de datos. Este script permite crear un entorno coherente de demostración con roles, usuarios, planes, modelos de inteligencia artificial, modos de ejecución y pipelines ya configurados.

Sin datos iniciales, sería necesario crear manualmente usuarios, planes, modelos, modos y pipelines antes de poder probar el flujo completo. Con el script de inicialización, la aplicación puede arrancar con un conjunto de datos preparado para demostrar los casos principales.

Entre los datos preparados se incluyen usuarios con distintos perfiles, modelos como YOLO y MediaPipe, modos como detección, recorte de personas, manos y pose, y pipelines de una o varias etapas. Esto permite probar tanto ejecuciones simples como flujos más complejos.

5.15. Síntesis

Los aspectos más relevantes del desarrollo se concentran en la evolución desde una demo de procesamiento de imágenes hasta una aplicación web modular y escalable funcionalmente. La demo inicial permitió validar la ejecución de modelos sobre imágenes enviadas desde el navegador. A partir de ahí, el proyecto incorporó arquitectura por capas, persistencia, autenticación, control de acceso, pipelines, administración de flujos, resultados temporales y pruebas automatizadas.

La decisión más importante ha sido evitar que los pipelines y los modelos quedasen fijados de forma rígida en el código. Gracias al modelo de datos, a la factoría de inteligencia artificial, a los launchers y a la zona de administración, la aplicación permite crear y modificar flujos de análisis con menor dependencia del código fuente. Esta característica es la que convierte el proyecto en una plataforma ampliable y no en una simple demo cerrada.

En conjunto, el trabajo desarrollado demuestra una integración completa de varias áreas de la ingeniería informática: desarrollo web, visión por computador, bases de datos, seguridad básica, arquitectura software, pruebas y documentación. La aportación principal no está únicamente en utilizar YOLO o MediaPipe, sino en construir una plataforma capaz de organizar, controlar y ampliar flujos de análisis visual de forma mantenible.

6. Trabajos relacionados

Este capítulo sitúa el proyecto desarrollado frente a otras herramientas y plataformas relacionadas con la ejecución de modelos de inteligencia artificial y la construcción de flujos de procesamiento de imágenes.

No se pretende realizar un estado del arte exhaustivo, sino comparar el TFG con tres referencias representativas: Gradio, Roboflow y Kubeflow Pipelines. Estas soluciones permiten ubicar el proyecto entre una demo sencilla de inteligencia artificial y una plataforma industrial de orquestación de modelos.

6.1. Herramientas y plataformas relacionadas

Gradio es una biblioteca orientada a construir interfaces web rápidas para modelos de aprendizaje automático [11]. Se relaciona con la primera fase del proyecto, en la que se buscaba validar el flujo básico de subir una imagen, ejecutar un modelo y mostrar un resultado visual. Sin embargo, Gradio está más orientado al prototipado rápido, mientras que este TFG desarrolla una aplicación web completa con autenticación, persistencia, permisos, pipelines, historial y administración.

Roboflow es una plataforma centrada en visión por computador. Roboflow Inference permite ejecutar modelos mediante una API HTTP, mientras que Roboflow Workflows permite construir flujos de procesamiento visual formados por varios bloques [28, 29]. Es la referencia más cercana al proyecto por la idea de workflows de visión por computador. La diferencia principal es que este TFG implementa una solución propia, en la que los pipelines se

modelan en una base de datos relacional y pueden gestionarse desde la zona de administración, sin modificar directamente el código fuente cuando se trabaja con modelos y modos ya disponibles.

Kubeflow Pipelines es una plataforma orientada a definir y ejecutar workflows de aprendizaje automático sobre infraestructuras basadas en contenedores y Kubernetes [15]. Comparte con este proyecto la idea de dividir un proceso complejo en etapas, pero su alcance es distinto. Kubeflow está pensado para entornos MLOps de mayor escala, mientras que este TFG adapta el concepto de pipeline a una aplicación web más ligera, basada en Flask y en una base de datos relacional.

6.2. Comparación con el proyecto desarrollado

La Tabla 6.1 resume la comparación entre las soluciones revisadas y el proyecto desarrollado.

Solución	Relación con el TFG	Diferencia principal
Gradio	Creación rápida de demos web para modelos de IA.	El TFG no se limita a una demo: incorpora usuarios, permisos, persistencia, pipelines, historial y administración.
Roboflow	Flujos de visión por computador e inferencia mediante servicios.	El TFG implementa una solución propia con pipelines administrables, YOLO, MediaPipe, planes, licencias y tokens temporales.
Kubeflow Pipelines	Orquestación de procesos de aprendizaje automático por etapas.	Kubeflow está orientado a MLOps y Kubernetes; el TFG adapta esa idea a una aplicación web más ligera.

Tabla 6.1: Comparación entre trabajos relacionados y el proyecto desarrollado.

6.3. Posicionamiento del TFG

A partir de esta comparación, el proyecto puede situarse en un punto intermedio entre una demo de inteligencia artificial y una plataforma industrial de orquestación. Frente a herramientas como Gradio, aporta una aplicación más completa. Frente a plataformas como Roboflow, implementa una solución propia y adaptada al alcance del TFG. Frente a Kubeflow Pipelines, toma la idea de procesamiento por etapas, pero sin introducir la complejidad de una infraestructura cloud-native.

La aportación principal del proyecto no está en crear un nuevo modelo de inteligencia artificial, sino en integrar modelos existentes dentro de una aplicación web modular. El sistema permite ejecutar modelos como YOLO y MediaPipe, organizar flujos mediante pipelines, controlar qué usuarios pueden acceder a ellos, registrar ejecuciones y gestionar resultados temporales mediante tokens.

Uno de los aspectos más diferenciadores es que los pipelines no quedan fijados como código cerrado. La aplicación permite que un administrador cree, modifique o elimine pipelines desde la propia interfaz, siempre que trabaje con modelos y modos ya registrados. Esta característica refuerza la escalabilidad funcional del sistema y lo diferencia de una simple demo de procesamiento de imágenes.

En síntesis, este TFG une varios elementos que suelen aparecer por separado: visión por computador, aplicación web, factoría de modelos, pipelines administrables, control de acceso, persistencia, resultados temporales y administración. Esa integración constituye el valor principal del trabajo desarrollado.

7. Conclusiones y líneas de trabajo futuras

En este capítulo se recogen las principales conclusiones obtenidas tras el desarrollo del Trabajo de Fin de Grado, así como las posibles líneas de evolución futura del proyecto.

El trabajo partía de una idea inicial concreta: desarrollar una aplicación web capaz de procesar imágenes mediante modelos de inteligencia artificial. Sin embargo, durante su desarrollo, esa idea evolucionó hacia una plataforma más completa, en la que no solo se ejecutan modelos de visión por computador, sino que también se gestionan usuarios, permisos, planes, modelos disponibles, pipelines, ejecuciones, archivos temporales y funciones de administración.

La principal aportación del proyecto no está en la creación de un nuevo modelo de inteligencia artificial, sino en la integración de modelos existentes dentro de una aplicación web modular, ampliable y controlada.

7.1. Conclusiones

El objetivo principal del proyecto se ha cumplido. Se ha desarrollado una aplicación web capaz de recibir imágenes, ejecutar sobre ellas modelos de inteligencia artificial y devolver resultados tanto visuales como estructurados. El usuario puede registrarse, iniciar sesión, consultar los pipelines disponibles, subir una imagen, ejecutar un análisis y visualizar los resultados obtenidos.

Uno de los resultados más importantes ha sido la evolución desde una demo inicial de procesamiento de imágenes hasta una aplicación con una

arquitectura más cercana a un sistema real. La primera fase permitió validar la integración de YOLO y MediaPipe desde un servidor web. A partir de esa base, el proyecto incorporó persistencia, autenticación, control de acceso, pipelines, historial, resultados temporales y administración.

Desde el punto de vista técnico, una de las decisiones más relevantes ha sido diseñar el sistema de forma modular. La separación entre controladores, servicios, modelos de datos, factoría de inteligencia artificial, launchers y vistas ha permitido reducir el acoplamiento entre las distintas partes de la aplicación. Gracias a ello, la lógica principal no depende directamente de una biblioteca concreta como YOLO o MediaPipe, sino que delega la ejecución en componentes especializados.

También destaca la incorporación de pipelines de procesamiento. Esta funcionalidad permite organizar el análisis de imágenes como una secuencia de etapas, donde cada etapa se asocia a un modelo y a un modo de ejecución. Esto aporta más flexibilidad que una ejecución aislada de modelos, ya que permite construir flujos simples o combinaciones más complejas, como detectar personas, generar recortes y aplicar después otro análisis sobre cada uno de ellos.

La escalabilidad funcional de la aplicación es otro de los puntos más relevantes del proyecto. Los pipelines no quedan fijados como código cerrado, sino que pueden gestionarse desde la zona de administración. Esto permite crear, modificar o eliminar flujos de procesamiento utilizando modelos y modos ya registrados, sin necesidad de alterar directamente el código fuente. Esta característica diferencia el sistema de una demo estática y lo acerca a una plataforma ampliable.

El modelo de datos también ha sido una parte fundamental. La base de datos no solo almacena usuarios, sino también roles, planes, suscripciones, alquileres, modelos de inteligencia artificial, modos, pipelines, etapas, ejecuciones y archivos temporales. Esta estructura permite representar la lógica del sistema de forma persistente y controlar la coherencia de los flujos definidos.

La gestión de resultados temporales mediante tokens de descarga ha permitido evitar la exposición directa de rutas internas del servidor. Además, la separación entre ejecuciones y archivos físicos facilita conservar trazabilidad sin obligar a almacenar indefinidamente todos los resultados generados.

Por último, la incorporación de pruebas automatizadas ha contribuido a mejorar la fiabilidad del sistema. Estas pruebas han permitido validar controladores, modelos de datos, servicios, factoría de inteligencia artificial,

launchers, pipelines y situaciones de error. En un proyecto que combina desarrollo web, base de datos, permisos y modelos de inteligencia artificial, esta validación resulta especialmente importante.

A nivel personal, el proyecto ha permitido aplicar de forma conjunta muchos de los conocimientos adquiridos durante el grado: desarrollo backend, bases de datos, arquitectura software, pruebas, seguridad básica, documentación técnica e integración de modelos de inteligencia artificial. También ha servido para comprender la diferencia entre construir una demo funcional y desarrollar una aplicación organizada, mantenible y preparada para crecer.

7.2. Líneas de trabajo futuras

Aunque el proyecto cumple los objetivos planteados, existen varias líneas de trabajo que permitirían mejorarlo y acercarlo a un entorno más real.

Una primera línea de mejora sería completar la contenerización del sistema mediante Docker. Esto facilitaría que la aplicación, la base de datos y las dependencias necesarias pudieran ejecutarse de forma reproducible en otros equipos. Esta mejora sería especialmente útil de cara a la defensa, al despliegue o a futuras pruebas por parte de terceros.

Otra mejora importante sería incorporar un sistema de migraciones de base de datos, por ejemplo mediante Alembic o Flask-Migrate. Actualmente, el sistema puede inicializar una base de datos para demostración, pero en un entorno real sería necesario evolucionar el esquema sin reinicializarlo completamente.

También sería conveniente añadir procesamiento asíncrono para los pipelines. En la versión actual, el procesamiento se realiza dentro del flujo de petición. Para imágenes grandes, modelos más pesados o pipelines con varias etapas, sería más adecuado utilizar una cola de tareas. De esta forma, el usuario podría lanzar una ejecución, consultar su estado y recuperar los resultados cuando estuvieran disponibles.

Otra línea futura sería ampliar el catálogo de modelos y modos. La arquitectura actual facilita esta evolución, ya que la incorporación de una nueva familia de modelos podría realizarse mediante un nuevo launcher y su registro en la factoría. También podrían añadirse nuevos modos sobre modelos ya existentes, como clasificación, segmentación, OCR o análisis de calidad de imagen.

La administración de pipelines también podría ampliarse. Aunque el sistema ya permite gestionar pipelines desde la aplicación, en el futuro podría

añadirse una interfaz más visual para construir flujos, reordenar etapas, validar compatibilidades o representar gráficamente la ejecución de cada pipeline.

En cuanto a la seguridad, sería recomendable reforzar la validación de ficheros subidos por el usuario, limitar tamaños máximos, controlar mejor los tipos MIME, registrar eventos relevantes y revisar de forma más exhaustiva los permisos aplicados en cada operación. Estas medidas serían necesarias si la aplicación se utilizara en un entorno con usuarios reales.

La tienda y los planes podrían evolucionar hacia un sistema comercial real. Actualmente validan la lógica funcional de acceso a modelos y pipelines, pero no integran pagos reales. Una evolución futura podría incorporar una pasarela de pago, gestión de facturación, webhooks, cancelaciones y control transaccional de suscripciones.

También podría mejorarse el almacenamiento de resultados. En lugar de guardar los archivos generados en el propio servidor, podrían almacenarse en un servicio externo de almacenamiento de objetos. Esto permitiría separar la aplicación del almacenamiento físico y aplicar políticas de expiración más avanzadas.

Finalmente, sería útil añadir documentación formal de los endpoints mediante OpenAPI o Swagger, así como monitorización básica del sistema. Estas mejoras facilitarían probar la aplicación, detectar errores y preparar el proyecto para un posible despliegue más profesional.

En conjunto, estas líneas futuras muestran que el proyecto puede seguir creciendo sobre la base desarrollada. La arquitectura modular, la factoría de modelos, los pipelines administrables y el modelo de datos permiten que la aplicación no quede cerrada a los casos implementados, sino preparada para incorporar nuevas funcionalidades de forma progresiva.

Bibliografía

- [1] Atlassian. Jira software documentation. <https://www.atlassian.com/software/jira/guides>, 2024.
- [2] Michael Bayer. Sqlalchemy documentation. <https://www.sqlalchemy.org/>, 2024.
- [3] DBeaver Corporation. Dbeaver documentation. <https://dbeaver.com/docs/dbeaver/>, 2024.
- [4] NumPy Developers. Numpy documentation. <https://numpy.org/doc/>, 2024.
- [5] Google AI Edge. Mediapipe solutions guide. <https://ai.google.dev/edge/mediapipe/solutions/guide>, 2024. [Online; Accedido 10-Diciembre-2025].
- [6] MariaDB Foundation. Mariadb knowledge base. <https://mariadb.com/kb/en/>, 2024.
- [7] Python Software Foundation. Python documentation. <https://docs.python.org/3/>, 2024.
- [8] Python Software Foundation. unittest.mock — mock object library. <https://docs.python.org/3/library/unittest.mock.html>, 2024.
- [9] GitHub. Github documentation. <https://docs.github.com/>, 2024.
- [10] Google. Gemini ai. <https://deepmind.google/technologies/gemini/>, 2024.
- [11] Gradio. Gradio documentation. <https://www.gradio.app/docs>, 2026.

- [12] Holistics. dbdiagram.io documentation. <https://docs.dbdiagram.io/>, 2024.
- [13] ECMA International. The json data interchange syntax. <https://www.ecma-international.org/publications-and-standards/standards/ecma-404/>, 2017.
- [14] JGraph. draw.io documentation. <https://www.drawio.com/doc/>, 2024.
- [15] Kubeflow. Kubeflow pipelines documentation. <https://www.kubeflow.org/docs/components/pipelines/overview/>, 2026.
- [16] Saurabh Kumar. python-dotenv documentation. <https://pypi.org/project/python-dotenv/>, 2024.
- [17] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Guang Yong, Juhyun Lee, Wan-Teh Chang, Wei Hua, Manfred Georg, and Matthias Grundmann. Mediapipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*, 2019.
- [18] Microsoft. Microsoft teams documentation. <https://learn.microsoft.com/microsoftteams/>, 2024.
- [19] Microsoft. Visual studio code documentation. <https://code.visualstudio.com/docs>, 2024.
- [20] Mozilla Developer Network. Css: Cascading style sheets. <https://developer.mozilla.org/en-US/docs/Web/CSS>, 2024.
- [21] Mozilla Developer Network. Html: Hypertext markup language. <https://developer.mozilla.org/en-US/docs/Web/HTML>, 2024.
- [22] Mozilla Developer Network. Javascript. <https://developer.mozilla.org/en-US/docs/Web/JavaScript>, 2024.
- [23] OpenAI. Chatgpt language model. <https://openai.com/chatgpt>, 2024.
- [24] Debian Project. Debian documentation. <https://www.debian.org/doc/>, 2024.
- [25] Pallets Projects. Flask documentation. <https://flask.palletsprojects.com/>, 2024.

- [26] Pallets Projects. Werkzeug documentation. <https://werkzeug.palletsprojects.com/>, 2024.
- [27] pytest developers. pytest documentation. <https://docs.pytest.org/>, 2024.
- [28] Roboflow. Roboflow inference documentation. <https://inference.roboflow.com/>, 2026.
- [29] Roboflow. Roboflow workflows documentation. <https://docs.roboflow.com/workflows>, 2026.
- [30] SQLite. Sqlite documentation. <https://www.sqlite.org/docs.html>, 2024.
- [31] OpenCV team. Opencv library documentation. <https://docs.opencv.org/>, 2024.
- [32] Landon Turner. Flask-jwt-extended documentation. <https://flask-jwt-extended.readthedocs.io/>, 2024.
- [33] Ultralytics. YOLOv8 docs. <https://docs.ultralytics.com/>, 2024.
- [34] Wikipedia. Latex — wikipedia, la enciclopedia libre. <https://es.wikipedia.org/w/index.php?title=LaTeX&oldid=84209252>, 2015. [Internet; descargado 30-septiembre-2015].